

AI Runtime 3D Generator

An LLM-driven system, built in Unity, that turns natural-language descriptions into real, structured 3D models — generated at runtime, during live gameplay, with no pre-made assets.

Type: Solo personal project (several weeks, ongoing) · **Role:** Sole designer, architect & developer · **Positioning:** AI / LLM Engineering (primary) → Software Engineering → Unity / Tools (secondary)

How to use this brief (for a résumé assistant): This is detailed source material on my most significant project. Use it to write or tailor résumé bullets for a given role — select the most relevant details, keep all claims accurate, and don't overstate (it is a working prototype, not a shipped product).

What it does

- You type a description — *"a Ford EcoBoost V6," "an aircraft carrier," "a windmill for a farm"* — and it constructs a real 3D model **live in the running game**, not in an offline editor step.
- Enables **dynamic, player-driven content**: objects generated on the fly from player input instead of being authored ahead of time.
- Produces recognizable results across wildly different domains — mechanical parts, engines, vehicles, buildings, ships, and infrastructure.

Architecture & how it works

- **Probabilistic / deterministic separation by design.** The LLM never generates geometry. It emits a compact, constrained JSON **"blueprint"** — a sequence of operations (`spawn` / `repeat` / `array`) over primitive forms, each with a bounding-box size and a position. A hand-written **deterministic C# engine** executes that blueprint into real meshes at runtime.
- **Two-pass agentic pipeline.** Pass one builds structure (geometry); pass two assigns materials and color (paint) — two constrained LLM calls bracketing a deterministic core. Runs on Claude Opus 4.8 with adaptive extended thinking.
- **Calibrated spatial contract.** Every shape obeys one unified rule (`size` = bounding box, `position` = center), verified against a reference "calibration sheet," keeping the model's output and the engine's interpretation in exact lockstep.
- **Zero external assets.** All geometry is generated at runtime from a small primitive set plus a self-made material palette — no model libraries, asset packs, or imported content.
- **Robustness.** Input/schema validation (rejects degenerate sizes, detects circular parent references), resilient JSON extraction from model output, deterministic processing, and error handling.

Detailed hierarchical decomposition

Every generation produces a single root object containing the model's **entire part tree**, broken down the way the real object is:

- **Each part is its own named GameObject**, labeled by function — e.g., an engine becomes `engine_block`, `cylinder_bank_left/right`, `valve_cover_left/right`, `intake_manifold`, `turbo_left/right` with their `downpipe` / `charge_pipe`, `crank_pulley`, `alternator`, `serpentine_belt`, `coil_left_1/2/3`, and more.
- **Composite elements are real sub-assemblies** — a window is a frame + glass; a building's windows and doors are auto-grouped into container nodes (`Windows`, `Doors`).

- Because every part is an **individually addressable object**, any single piece can be selected, moved, rotated, recolored, or deleted — *and* targeted in code (animate a piston, swing a door, swap a turbo, attach physics or gameplay logic to a named part). The output is a **programmable, inspectable object graph**, not a static mesh.

Asset lifecycle & dual workflows

The key enabling property: **every asset is simultaneously a real, editable Unity hierarchy and a portable JSON blueprint (pure data)**. That duality unlocks the full lifecycle:

- **Save** any generated asset to a searchable on-disk library (blueprint + metadata).
- **Re-spawn deterministically** — the same blueprint always rebuilds the identical model, so saved assets are reproducible forever.
- **Duplicate** any asset instantly.
- **Hand-edit** — refine the JSON blueprint directly, or edit individual parts in-scene (move / scale / recolor / remove pieces).
- **Refine & iterate** — regenerate from a sharpened prompt, tweak by hand, then re-save the improved version.
- **Build a permanent library** — curate a growing set of AI-generated, human-refined assets you own and reuse like any other game content, indefinitely.

Workflow A — Author / library mode: generate, curate, fine-tune, and bank a reusable asset library for a project.

Workflow B — Runtime / in-game mode: generate content live, on demand, from player input while the game is being played.

The same engine powers both — a content-creation pipeline *and* a live gameplay capability.

Key technical accomplishments

- Designed an **LLM-to-structured-output architecture** that makes generative results reproducible, verifiable, and unit-testable by constraining the model to a typed blueprint instead of free-form geometry.
- Engineered a **calibrated prompt ↔ engine contract** that holds across a universal domain (a bolt to a battleship) — solving the core challenge of consistency in an open-ended builder.
- Built a **two-pass agentic pipeline** integrating Claude Opus 4.8 with adaptive extended thinking.
- Implemented a **deterministic geometry engine** (~14 primitive / composite shapes, bounding-box fitting, named hierarchical output) with **EditMode unit tests** on the core math.
- Built a full **data-driven asset lifecycle** (save, deterministic re-spawn, duplicate, hand-edit, refine into a persistent library) and a **self-wiring, code-built runtime UI** with interaction and save/load.

Tech stack

Languages / Engine: C#, Unity 6.3 LTS · **AI:** Anthropic Claude API — Claude Opus 4.8 (adaptive extended thinking) · **3D / Rendering:** ProBuilder (runtime procedural mesh generation), Universal Render Pipeline, TextMeshPro · **Data / Tooling:** Newtonsoft JSON, Unity Test Framework (EditMode), Git

Skills demonstrated

LLM application development · AI engineering · prompt engineering · structured / constrained generation · JSON schema · agentic / multi-step LLM pipelines · Anthropic Claude API · extended / adaptive thinking · deterministic systems design · reproducibility · software architecture · separation of concerns · C# · Unity · ProBuilder · runtime procedural generation · computational geometry / 3D math · unit testing (Unity Test Framework) · Git · UI / UX

Scope & honest status

- Single descriptions yield models of **dozens of organized parts** (e.g., a generated V6 engine $\approx$ 30+ labeled components).
- Demonstrated range: a single bolt $\rightarrow$ a labeled engine $\rightarrow$ buildings $\rightarrow$ an **aircraft carrier**, a **naval fleet**, and a **SpaceX-style launch complex**.
- **Status:** working proof-of-concept. Geometry is blockout / prototype quality (clean primitive-based meshes); generation takes seconds to a couple of minutes; solo-built. Best framed as a strong prototype demonstrating AI-systems architecture and engineering judgment — not a shipped product or novel research.

Ready-to-use résumé bullets

- **Designed and built an LLM application (Unity, C#, Claude API) that generates structured, editable 3D models from natural language at runtime**, architecting a deterministic engine behind a constrained-output model so the same prompt reproducibly yields the same verifiable result.
- **Engineered a calibrated prompt-to-engine contract and a two-pass agentic pipeline** (build $\rightarrow$ paint) that keep model output and engine interpretation in exact lockstep, achieving consistent results across an open-ended domain (mechanical parts to full vehicles and structures).
- **Decomposed each generated model into a detailed, named object hierarchy** (functionally-labeled parts and auto-grouped sub-assemblies), producing a programmable object graph where any part can be selected, edited, or driven by gameplay code.
- **Built a data-driven asset lifecycle** — generated objects are both editable Unity hierarchies and portable JSON, enabling save, deterministic re-spawn, duplication, hand-editing, and refinement into a persistent, reusable asset library.
- **Implemented a deterministic geometry engine with ~14 procedural primitives and EditMode unit tests**, plus a fully code-built runtime UI — generating everything from a single part to an aircraft carrier with zero imported assets.

Project brief — AI Runtime 3D Generator. Source material for résumé generation; all claims accurate to the working prototype.